

IA Aplicada com LLMs

Aula 03 — Anatomia do prompt e as técnicas

Onde paramos

A Aula 02 deixou quase tudo sob controle:

-  modelo escolhido com critério
-  parâmetros ajustados por tipo de tarefa
-  saída em JSON garantida por schema

Sobrou uma coisa sem controle nenhum:

o texto

E o texto é a parte que todo mundo acha que já sabe fazer

Os conselhos que circulam por aí:

"seja mais específico" · "dê contexto" · "peça para pensar passo a passo"

Não estão errados. Também não são **engenharia**:

não dizem **quando** aplicar, **quanto custa**, nem **como saber se funcionou**.

Um aviso de posicionamento

A Aula 00 foi explícita: **este não é um curso de prompt engineering.**

"Prompt é meio, não fim."

O que interessa aqui é o **critério de escolha** — e, com a mesma força, saber quando a resposta **não é** um prompt melhor, e sim outra arquitetura.

Roteiro

1. **O prompt como contrato** — as peças e o que elas garantem
2. **Zero-shot e few-shot** — e o que os exemplos realmente ensinam
3. **Chain-of-thought e self-consistency** — quando pensar em voz alta paga
4. **As demais** — knowledge, chaining, ToT, meta-prompting
5. **A tabela de decisão** — e quando **não** usar cada uma

Objetivos

- **Distinguir** system, role e contextual prompting
- **Escrever** um output contract, com **exemplos negativos**
- **Explicar** por que zero-shot funciona
- **Aplicar** few-shot sabendo o que ele ensina de verdade
- **Escolher** técnica pela **tarefa e pelo modelo**
- **Reconhecer quando cada técnica não ajuda**

Parte 1

O prompt como contrato

"Seja mais específico" se autodestrói

Se mais específico é sempre melhor, o prompt ideal seria **infinito**.

Não é:

- cada token **custa dinheiro**
- cada token **come janela**
- e, passado certo ponto, **piora** o resultado (*lost in the middle*)

O conselho útil tem três partes

Específico sobre a **tarefa**, o **formato** e os **casos de fronteira**.
Não sobre tudo.

O ruim

Analise cuidadosamente e com muita atenção a mensagem do cliente abaixo, sendo bastante detalhado e específico na sua análise, e me diga do que se trata.

Mais palavras, menos informação.

"Cuidadosamente", "com muita atenção", "detalhado" **não restringem nada** — o modelo não tem um botão de esforço que essas palavras acionam.

O bom

Classifique a mensagem em: entrega_atrasada, endereco_errado, produto_avariado, duvida, elogio.

Se houver mais de um problema, escolha o da reclamação.
Se não houver problema, use 'duvida' ou 'elogio'.

Responda apenas o rótulo.

Fecha o **espaço de respostas**: os rótulos possíveis, a ambiguidade resolvida, o formato definido.

A regra de trabalho

Cada frase do prompt deve **eliminar** alguma possibilidade.

Apague a frase. Não consegue apontar o que ela impedia?

É enfeite — e enfeite você paga em toda requisição.

Três camadas, três ciclos de vida

Camada	Define	Muda quando
System	comportamento, restrições, formato	~nunca (versionado, testado)
Role	papel: perspectiva, vocabulário, tom	por produto ou público
Contextual	informação desta chamada	a cada requisição

Confundi-las é a origem de metade dos problemas.

Role prompting: o que ele faz

"Você é um especialista de classe mundial em..."

Provavelmente a linha mais escrita da história do prompt engineering.

O que ela realmente muda:

- registro e **vocabulário**
- o que o modelo **assume que você já sabe**
- convenções de **formato** de uma profissão
- o **enquadramento** — jurídico × comercial × operação

E o que ela **não** muda

- a **acurácia** em fatos
- a **capacidade de raciocínio**

Dizer "você é um especialista" não dá ao modelo conhecimento que ele não tem, nem faz com que ele calcule melhor.

Ele já viu texto de especialista no treino. O papel escolhe **de qual parte dessa distribuição ele puxa** — não eleva o teto.

Role é um seletor de registro, não um multiplicador de qualidade.

Aplique a regra da §1

✗ Você é um especialista de classe mundial em logística.

O que essa frase **impede**? Nada que você consiga nomear.

✓ Escreva para um analista de logística.
Use os termos do setor (coleta, redespacho, avaria, extravio)
sem explicá-los.
Não sugira ações que dependam de sistemas que você não conhece.
Se faltar dado para concluir, diga o que falta.

Três frases, três possibilidades eliminadas — **e sem precisar do papel.**

Se você consegue escrever o que quer sem o papel, escreva.
Use o papel quando o **enquadramento** for o objetivo.

O system vai junto sempre

Tudo o que você escreve no system prompt é reenviado **em toda requisição**.

Ele é o lugar onde frase-enfeite mais pesa: ocupa janela que, num sistema real, faria falta para o histórico ou para os documentos.

Por isso a regra da §1 vale em dobro aqui:

se a frase não elimina uma possibilidade, ela não deveria estar no system.

Delimitadores

```
{"role": "user", "content": f""  
{INSTRUCAO}  
  
<mensagem>  
{mensagem}  
</mensagem>  
""}
```

Para o modelo, **tudo é uma sequência de tokens**. Não existe canal de instrução e canal de dado — a fronteira é apenas textual.

⚠ Delimitadores **ajudam o modelo**. Não **garantem** nada — e é por isso que prompt não é mecanismo de segurança (aula de segurança).

O output contract

O schema garante o **formato**. O contrato cobre o que o schema não cobre:

Parte	Exemplo
Formato	schema JSON, ou "apenas o rótulo"
Domínio	os 5 rótulos possíveis (<code>enum</code>)
Regras de fronteira	"com dois pedidos, vale o reclamado"
Exemplos negativos	"não escreva 'Claro! Aqui está'"

Exemplos negativos

A parte que quase ninguém escreve — e a que mais economiza depuração.

Responda APENAS o rótulo, em minúsculas.
Não escreva explicação, saudação ou pontuação final.
Não repita a mensagem do cliente.

O modelo tem tendências fortes: cumprimentar, explicar o que vai fazer, oferecer ajuda no fim. **Dizer "não faça X" é mais eficaz que esperar que ele adivinhe pelo silêncio.**

Parte 2

As técnicas

Zero-shot

```
Classifique o texto em neutro, negativo ou positivo.  
Texto: Acho que as férias estão boas.  
Sentimento:
```

Nenhum exemplo. Funciona.

Mas por quê? Um modelo treinado para prever o próximo token não tem motivo especial para *obedecer* a uma instrução.

Porque alguém pagou para funcionar

Instruction tuning (FLAN, Wei et al. 2022): depois do pré-treino, o modelo é ajustado em milhares de tarefas **descritas como instruções**.

Ele aprende o padrão: *"quando o texto tem cara de instrução, o que vem depois é a execução dela"*.

Some **RLHF**, e você tem o modelo instruct da Aula 02.

Zero-shot não é propriedade natural de LLM. É **produto de engenharia**.

Comece sempre por zero-shot

É o mais barato.

Em tarefas comuns, **ele resolve**.

Só escale a técnica quando **medir** que precisa.

Few-shot

Uma "whatpu" é um animal pequeno e peludo nativo da Tanzânia.
Um exemplo de frase que usa a palavra whatpu é:
Estávamos viajando pela África e vimos esses whatpus muito fofos.

"Farduddle" significa pular para cima e para baixo bem rápido.
Um exemplo de frase que usa a palavra farduddle é:

In-context learning: aprendizado na janela, **sem alterar peso nenhum.**

Emergiu com a escala (Brown et al., 2020).

O que os exemplos ensinam?

A intuição de todo mundo:

"o modelo aprende dos exemplos o mapeamento entrada $\rightarrow$ saída"

Min et al. (2022) testaram isso do jeito mais direto possível.

Eles embaralharam os rótulos

```
This is awesome!      // Negative      ← errado de propósito
This is bad!          // Positive       ← errado de propósito
Wow that movie was rad! // Positive
What a horrible show!  //
```

Saída: **Negative**

A resposta **correta** — apesar de os exemplos ensinarem o contrário.

A conclusão dos autores

*"O espaço de rótulos e a distribuição do texto de entrada especificados pelas demonstrações são ambos importantes — **independentemente de os rótulos estarem corretos para as entradas individuais.**"*

O que o few-shot ensina de verdade

1. **Quais rótulos existem** (o espaço de saída)
2. **Que tipo de texto** entra
3. **O formato** da resposta

O mapeamento é a **menor** parte.

Os três números

Configuração	Acurácia	O modelo tinha
Few-shot correto	85%	espaço + distribuição + formato + mapeamento
Few-shot embaralhado	82%	espaço + distribuição + formato
Sem exemplos	61%	nada disso

85 → 82 = o que o **mapeamento** vale.

82 → 61 = o que **todo o resto** vale.

⚠ Não conclua "*exemplos não servem, pode remover*" com só dois números.

O que fazer com isso

- cubra todos os rótulos, principalmente os raros
- use entradas parecidas com as reais — com erro de digitação
- formato rigorosamente igual em todos os exemplos
- não gaste horas caçando os exemplos "perfeitos"

E lembre: os exemplos **vão junto em toda requisição**, ocupando janela.

Por isso a regra é **cobertura, não quantidade**: 3 exemplos bem escolhidos valem mais que 10 redundantes.

Onde o few-shot não salva

Os ímpares deste grupo somam um número par: 15, 32, 5, 13, 82, 7, 1.
R:

"Sim, os ímpares somam 107, que é par."

Duas coisas erradas: a soma é **41**, e 107 nem é par.

Acrescente 4 exemplos resolvidos: continua errado.

Por quê

Few-shot mostra o formato da resposta, não o caminho até ela.

Quando a tarefa exige etapas intermediárias, o modelo continua tentando saltar direto para o resultado — agora com a formatação certa.

Chain-of-thought

Os exemplos passam a mostrar **o raciocínio**, não só a resposta:

Os ímpares deste grupo somam um par: 4, 8, 9, 15, 12, 2, 1.
R: Os ímpares são 9, 15 e 1. Somando: $9 + 15 = 24$, $24 + 1 = 25$.
25 é ímpar. A resposta é Falso.

Ganho grande em aritmética, senso comum e problemas simbólicos.

Por que funciona

O modelo gera **um token por vez**, condicionado a tudo que já escreveu.

Ao escrever os passos, ele **coloca no próprio contexto** os resultados parciais de que vai precisar.

O raciocínio escrito é memória de trabalho externalizada.

Sem isso, ele teria que fazer a conta toda numa passagem só.

Zero-shot CoT

Kojima et al. (2022): boa parte do ganho sai **sem exemplo nenhum**.

Vamos pensar passo a passo.

Uma linha. Sem exemplos ocupando a entrada.

A melhor relação benefício/esforço da aula.

Quando **não** usar CoT

Tarefa de um passo — classificação, extração, tradução.

Não há etapas para escrever; o raciocínio não tem o que fazer.

E dá ao modelo espaço para **se contradizer antes de responder**.

Modelo de raciocínio — já faz isso internamente. Pedir é redundante.

⚠ E um detalhe prático: a resposta fica **longa**. Com `max_tokens` apertado, o raciocínio é cortado antes da conclusão — você recebe o caminho sem o destino (Aula 02, nota 03, §2.2).

Regra: CoT em tarefa de **várias etapas**.

No resto, prompt direto com um bom contrato de saída.

Self-consistency

Se o raciocínio é **amostrado**, ele varia — e um caminho errado leva a uma resposta errada.

```
prompt [→ raciocínio 1 → 254
        → raciocínio 2 → 254    → voto majoritário → 254
        → raciocínio 3 → 270
        → raciocínio 4 → 254
        → raciocínio 5 → 254
```

Caminhos errados erram de formas **diferentes**; o certo **converge**.

Repare no que mudou aqui

O **prompt é exatamente o mesmo** do zero-shot CoT.

A técnica não está no texto — está em **pedir várias respostas e comparar**.

É a primeira vez na aula em que a melhoria não vem de escrever melhor, e sim de **medir mais**.

⚠ E ela tem um limite: só ajuda quando o modelo **erra por variação**.

Se ele erra sempre igual, cinco execuções erradas dão uma maioria errada.

Generated knowledge

```
1ª chamada: "Liste fatos relevantes sobre <tema>."  
2ª chamada: "Dados estes fatos: <fatos>, responda: <pergunta>"
```

Ajuda quando o modelo "sabe" o fato mas não o traz ao responder direto.

 **O conhecimento sai do próprio modelo** — pode ser inventado com a mesma fluência. **Isto não resolve alucinação.**

Fato externo pede **RAG**.

Prompt chaining

Uma tarefa por chamada, a saída de uma alimentando a próxima.
(Vocês já fizeram isso no exercício 02: extrair, depois redigir.)

	Uma chamada	Cadeia
Depuração	tudo ou nada	você vê onde quebrou
Controle	nenhum no meio	valida entre etapas
Parâmetros	um conjunto	um por etapa

O ganho não é qualidade. É **observabilidade e controle**.

Tree of Thoughts

Em vez de **uma** linha de raciocínio, uma **árvore** de pensamentos parciais que o modelo avalia e por onde uma busca avança **e retrocede**.

Para problemas com exploração e *backtracking*.

Pesado: dezenas de chamadas e um laço de controle escrito por você.

A versão completa é, na prática, um **agente**. Ela reaparece com outro nome na aula de agentes (padrão *planning*).

Meta-prompting

Em vez de exemplos **concretos**, o **esqueleto** da solução — a forma do problema e da resposta, sem os detalhes.

- **eficiência de tokens** — estrutura ocupa menos que exemplos
- funciona como zero-shot, sem depender da escolha dos exemplos

⚠ Assume que o modelo **já conhece a tarefa**. Em domínio novo, degrada.

A tabela de decisão (1/2)

Técnica	O que você escreve	Quando usar	Quando não
Zero-shot	só a instrução	ponto de partida	—
Few-shot	+ 2 a 5 exemplos	formato, rótulos raros	se o schema já resolve
CoT	+ exemplos com raciocínio	várias etapas	tarefa de 1 passo
Zero-shot CoT	+ <i>"passo a passo"</i>	idem, sem exemplos	modelo de raciocínio
Self-consistency	nada — muda o n	erra por variação	se erra sempre igual

A tabela de decisão (2/2)

Técnica	O que você escreve	Quando usar	Quando não
Gen. knowledge	2 prompts encadeados	senso comum	fato externo → RAG
Chaining	1 prompt por etapa	etapas distintas	se 1 chamada resolve
ToT	um laço de busca	exploração	quase sempre
Meta-prompting	o esqueleto	tarefa conhecida	domínio novo

Como usar a tabela

De cima para baixo, parando no primeiro que resolve.

Escalar técnica sem medir é o mesmo erro de escolher o modelo maior sem testar o menor.

A ressalva de 2026

Tudo isso foi desenvolvido para modelos **instruct**.

Técnica	Instruct	Raciocínio
CoT / zero-shot CoT	ganho grande	redundante — pode atrapalhar
Self-consistency	ganho real	menor — já explora internamente
Few-shot	útil	útil para formato
Output contract	essencial	essencial

A técnica certa depende do modelo, não só da tarefa.

E trocar de modelo pode **invalidar o seu prompt** (deck 03).

Síntese (1/2)

- *"Seja mais específico"* não é conselho. **Cada frase deve eliminar uma possibilidade** — se não elimina, é enfeite que você paga sempre
- Três camadas: **system** (versionado), **role** (a mais superestimada), **contextual** (a que cresce)
- **Delimitadores ajudam o modelo, não garantem nada**
- **Output contract**: formato, domínio, fronteiras e **exemplos negativos**
- **Zero-shot funciona por instruction tuning**, não por natureza

Síntese (2/2)

- **Few-shot ensina espaço de rótulos, distribuição e formato — não o mapeamento.** Cubra as classes raras, use entradas realistas, formato idêntico
- **CoT externaliza a memória de trabalho.** "Pense passo a passo" captura boa parte do ganho por uma linha
- **Self-consistency** vota entre N raciocínios — o prompt é o mesmo; muda o `n`
- **Generated knowledge não cura alucinação** — fato externo pede RAG
- **Chaining** ganha em controle, não em qualidade
- **A tabela se lê de cima para baixo, parando no primeiro que resolve** — e a técnica certa **depende do modelo**